Evaluation Metrics:

1. Define confusion matrices, True Positives, False Positives, True Negatives, and False Negatives.

   Confusion Matrix:
   A confusion matrix is a table used to evaluate the performance of a classification model by comparing **actual class labels** with **predicted class labels**.

   Key Terms:

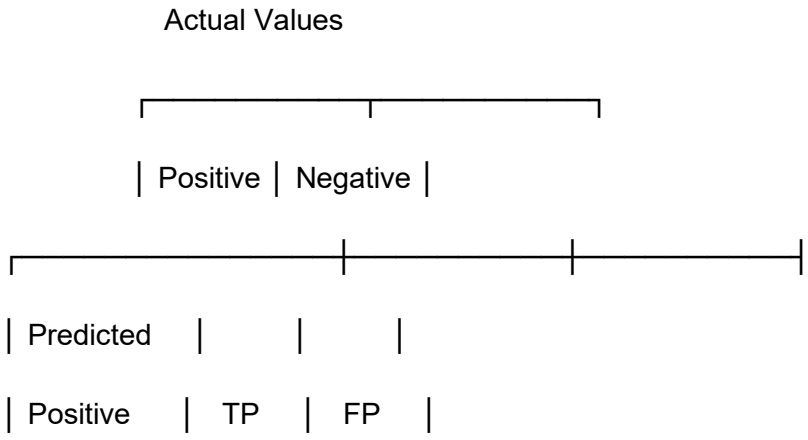
- **True Positive (TP):** The model correctly predicts the positive class.
- **False Positive (FP):** The model predicts positive when the actual class is negative (Type I error).
- **True Negative (TN):** The model correctly predicts the negative class.
- **False Negative (FN):** The model predicts negative when the actual class is positive (Type II error).

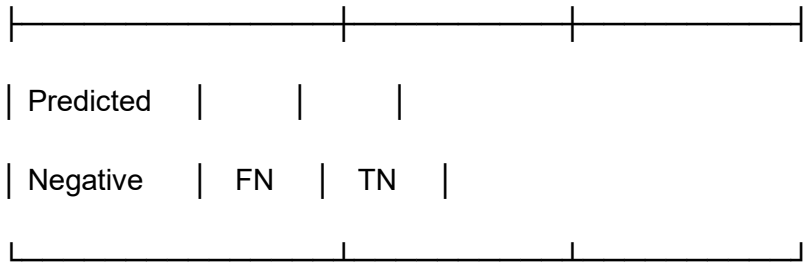
  Confusion Matrix

  A **tabular layout** used to evaluate the performance of a classification model by comparing predicted labels against actual (true) labels. It provides a **detailed breakdown** of correct and incorrect predictions across all classes.

  Binary Classification Example (Positive/Negative)

  text





Core Components Defined

1. True Positive (TP)

- **Prediction**: Positive
- **Actual**: Positive
- **Interpretation**: **Correctly identified** positive cases.
- **Example (Medical Test)**: Patient has disease → test correctly says "diseased".

2. False Positive (FP) - Type I Error

- **Prediction**: Positive
- **Actual**: Negative
- **Interpretation**: **Incorrectly flagged** as positive (false alarm).
- **Example**: Healthy patient → test incorrectly says "diseased".

3. True Negative (TN)

- **Prediction**: Negative
- **Actual**: Negative
- **Interpretation**: **Correctly identified** negative cases.
- **Example**: Healthy patient → test correctly says "healthy".

4. False Negative (FN) - Type II Error

- **Prediction**: Negative
- **Actual**: Positive
- **Interpretation**: **Missed positive** cases (failure to detect).
- **Example**: Patient has disease → test incorrectly says "healthy".

Practical Example: Spam Email Detection

Actual vs Predicted	Meaning	Consequence
TP (Spam → Predicted Spam)	Spam correctly filtered	User doesn't see sp
FP (Ham → Predicted Spam)	Legitimate email marked as spam (false alarm)	User misses importa email
TN (Ham → Predicted Ham)	Legitimate email correctly delivered	User sees normal emails
FN (Spam → Predicted Ham)	Spam slips to inbox (missed detection)	User sees unwanted spam

---

Why These Metrics Matter

Different **performance metrics** are derived from these four values:

- **Accuracy** = (TP + TN) / Total → Overall correctness
- **Precision** = TP / (TP + FP) → Quality of positive predictions
- **Recall/Sensitivity** = TP / (TP + FN) → Ability to find all positives

- **Specificity** = TN / (TN + FP) → Ability to find all negatives

  The **cost of FP vs FN** varies by application (e.g., in medical diagnosis, FN may be more dangerous than FP).

2. Explore metrics like Accuracy, Precision, Recall, ROC, and Area under the curve.

Classification Performance Metrics:

- **Accuracy:**
  Measures the overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Best when classes are balanced.

- **Precision:**
  Indicates how many predicted positives are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Important when false positives are costly (e.g., spam detection).

- **Recall (Sensitivity / True Positive Rate):**
  Measures how many actual positives are correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Crucial when missing positives is risky (e.g., disease detection).

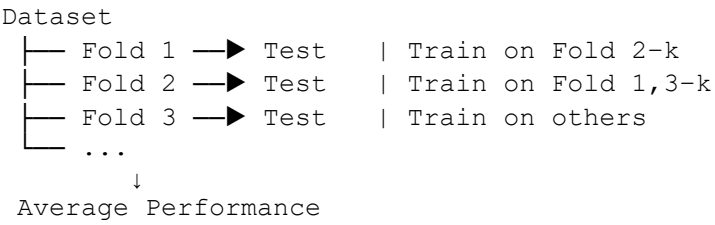
- **ROC (Receiver Operating Characteristic) Curve:**
  A graphical plot showing the trade-off between **True Positive Rate (Recall)** and **False Positive Rate** at different classification thresholds.
- **Area Under the Curve (AUC):**
  Represents the model's ability to distinguish between classes.
  - **AUC = 1.0:** Perfect classifier
  - **AUC = 0.5:** No better than random guessing

👉 **Summary:** Accuracy gives a broad view, Precision and Recall focus on error types, while ROC and AUC evaluate performance across all thresholds.

3. Explain cross-validation and its significance.

Cross-validation is a technique used to evaluate a model's performance by dividing the dataset into multiple parts, training the model on some parts and testing it on the remaining ones.
It provides a **reliable estimate of model generalization** and helps detect **overfitting**.

Simple Diagram (k-Fold Cross-Validation)

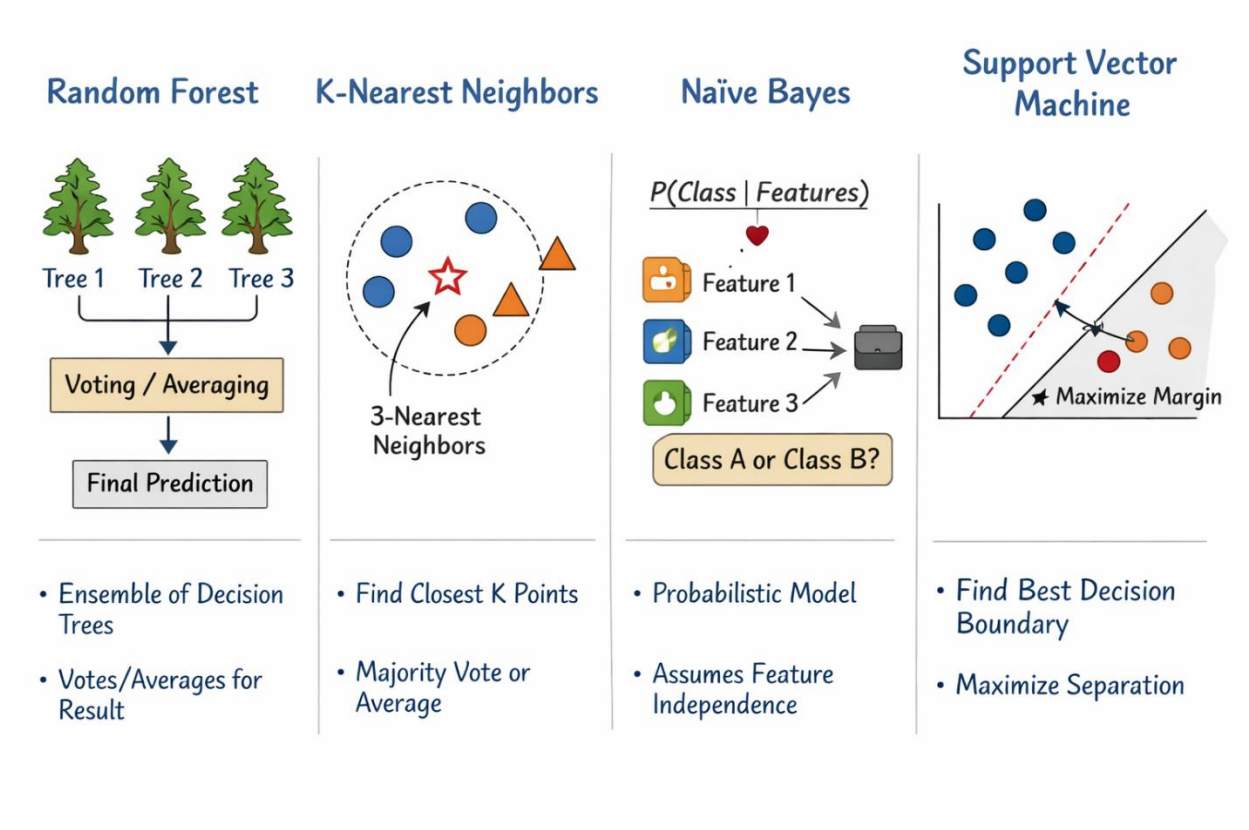


Key significance: better accuracy estimation, efficient data usage, and robust model selection.

Differentiate between regression metrics, highlighting the distinctions between MAE and MSE.

Main Supervised Classification Models:

1. Provide concise yet comprehensive explanations for Linear Regression, Logistic Regression, K-nearest neighbour, Support Vector Machine, Naïve Bayes, Decision Trees, Random Forests, Neural Networks, and Deep Neural Networks.



An **ensemble of decision trees** is a machine learning approach where **multiple decision trees are built and combined** to make a single, more accurate and robust prediction.

Core idea:
Instead of relying on one tree (which may overfit or be unstable), the ensemble **aggregates the predictions** of many trees—typically by **majority voting** (classification) or **averaging** (regression).

Why it works:

- Reduces overfitting
- Improves prediction accuracy
- Increases model stability

Common types of decision-tree ensembles:

- **Bagging (Bootstrap Aggregating):**
  Trains trees on different random samples of the data (e.g., **Random Forest**).

- **Boosting:**
  Trains trees sequentially, focusing more on previously misclassified samples (e.g., **AdaBoost**, **Gradient Boosting**, **XGBoost**).

- **Random Forest:**
  Combines bagging with random feature selection at each split.

👉 In short, an ensemble of decision trees leverages the **collective wisdom of many trees** to outperform a single decision tree.

Association Rules:

1. Define association rules and their applications.

Association Rules

$$Rule\ X \Longrightarrow Y$$

$$Support = \frac{Frequency\ (X,Y)}{N}$$

$$Confidence = \frac{Frequency\ (X,Y)}{Frequency(X)}$$

$$Lift = \frac{Support}{Support(X) * Support(Y)}$$

Definition

Association rule mining is a data mining technique used to discover **interesting relationships, correlations, or patterns** among items in large datasets.

An **association rule** is an implication of the form:

$$X \rightarrow Y$$

where:

- **X** and **Y** are itemsets
- $X \cap Y = \emptyset$

Meaning: If itemset **X** occurs, itemset **Y** is likely to occur.

Measures of Association Rules

1. **Support**
   Indicates how frequently an itemset appears in the database.

$$Support(X \rightarrow Y) = P(X \cup Y)$$

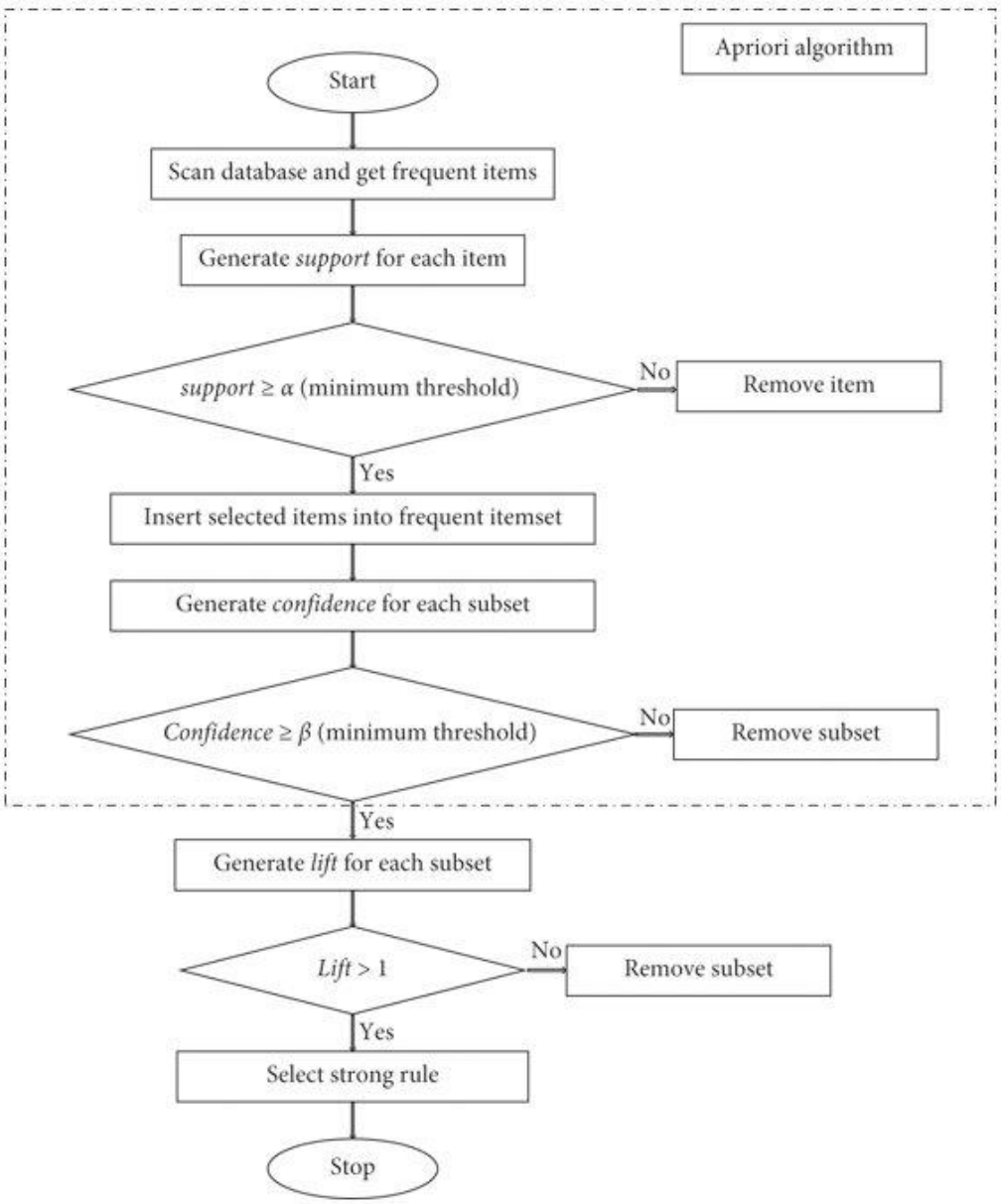
2. **Confidence**
   Indicates the reliability of the rule.

$$Confidence(X \rightarrow Y) = P(Y \mid X)$$

3. **Lift**
   Measures the strength of a rule compared to random occurrence.

$$Lift = \frac{Confidence(X \rightarrow Y)}{Support(Y)}$$

Association Rule Mining Process



Applications of Association Rules

1. **Market Basket Analysis**
    - Identifies products frequently bought together
    - Helps in product placement and promotions
    - Example: *Bread* → *Butter*
2. **Retail and E-commerce**
    - Recommendation systems
    - Cross-selling and up-selling strategies
3. **Healthcare**
    - Identifying relationships between symptoms and diseases
    - Drug interaction analysis
4. **Web Usage Mining**
    - Analyzing user navigation patterns
    - Improving website structure
5. **Fraud Detection**
    - Detecting unusual combinations of transactions
6. **Telecommunication**
    - Customer usage pattern analysis
    - Churn prediction

Association Rule Diagram (Text Representation)

Transaction Database

↓

Frequent Itemset Mining

↓

Association Rule Generation

↓

Strong Rules (High Support & Confidence)

Example

Rule:

$$\{Milk, Bread\} \rightarrow \{Butter\}$$

Interpretation:
Customers who buy milk and bread are likely to buy butter.

---

2. Explain the concepts of Antecedent and Consequent in a rule.

Antecedent and Consequent in an Association Rule
In **association rule mining**, a rule is written as:

$$X \rightarrow Y$$

where **X** is the **Antecedent** and **Y** is the **Consequent**.

---

1. Antecedent
The **antecedent** is the **"IF" part** of the rule.

- Appears on the **left-hand side (LHS)**
- Represents a set of items or conditions already present
- Acts as the **cause or condition**

Example:
Rule:

$$\{Milk, Bread\} \rightarrow \{Butter\}$$

Antecedent = **{Milk, Bread}**

---

2. Consequent
The **consequent** is the **"THEN" part** of the rule.

- Appears on the **right-hand side (RHS)**
- Represents items likely to occur with the antecedent
- Acts as the **effect or result**

Example:
Rule:

$$\{Milk, Bread\} \rightarrow \{Butter\}$$

Consequent = **{Butter}**

Relationship with Confidence

- **Confidence** measures how often the consequent occurs when the antecedent occurs:

$$Confidence(X \rightarrow Y) = P(Y \mid X)$$

High confidence means the consequent is very likely given the antecedent.

Simple Diagram (Text)
Antecedent (IF) Consequent (THEN)
Milk, Bread ⟶ Butter

Key Points for Exams

- Antecedent = **IF part (LHS)**
- Consequent = **THEN part (RHS)**
- Both are **disjoint itemsets**
- Rule format: **X → Y**

3. Introduce the Apriori Algorithm and define associated metrics: Support, Confidence, and Lift.

Apriori Algorithm and Associated Metrics

$$Rule\ X \Longrightarrow Y \begin{cases} Support = \frac{Frequency\ (X,Y)}{N} \\ Confidence = \frac{Frequency\ (X,Y)}{Frequency(X)} \\ Lift = \frac{Support}{Support(X) * Support(Y)} \end{cases}$$

Apriori Algorithm — Introduction

The **Apriori algorithm** is a classic algorithm used in **association rule mining** to discover **frequent itemsets** and generate **strong association rules** from large transaction databases.

It is based on the **Apriori principle**:

If an itemset is frequent, then all of its non-empty subsets must also be frequent.

This principle helps reduce the search space and improves efficiency.

Steps of the Apriori Algorithm

1. **Generate candidate 1-itemsets (C_1)**
2. **Prune** itemsets whose support is less than minimum support
3. Form **frequent itemsets (L_1)**

4. Generate candidate itemsets $\mathbf{C_k}$ from $\mathbf{L_{k-1}}$
5. Repeat pruning until no more frequent itemsets are found
6. Generate **association rules** from frequent itemsets

Apriori Algorithm Flow (Diagram – Text)

Transaction Database

↓

Candidate Itemsets (Ck)

↓

Support Counting

↓

Pruning (min support)

↓

Frequent Itemsets (Lk)

↓

Association Rules

Associated Metrics

1. Support

Support indicates how frequently an itemset appears in the database.

$$Support(X) = \frac{\text{Number of transactions containing } X}{\text{Total number of transactions}}$$

Support of a rule:

$$Support(X \rightarrow Y) = Support(X \cup Y)$$

---

2. Confidence

Confidence measures the **reliability** of an association rule.

$$Confidence(X \rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

It shows how often **Y** occurs when **X** occurs.

---

3. Lift

Lift measures the **strength** of a rule compared to random chance.

$$Lift(X \rightarrow Y) = \frac{Confidence(X \rightarrow Y)}{Support(Y)}$$

- Lift > 1 → Positive correlation
- Lift = 1 → No correlation
- Lift < 1 → Negative correlation

Example

Rule:

$$\{Milk, Bread\} \rightarrow \{Butter\}$$

- High **support** → Rule is common

- High **confidence** → Rule is reliable
- High **lift** → Rule is meaningful

Reinforcement Learning:

1. Briefly introduce the concepts (no need to include the formula) of Markov Decision Processes, the Bellman equation, temporal difference, exploration and exploitation, and deep Q learning.
   Markov Decision Process (MDP)
   A **mathematical framework** for modeling sequential decision-making where outcomes are partly random and partly controlled. It consists of:

- **States**: Possible situations the agent can be in.
- **Actions**: Choices available to the agent.
- **Transitions**: How states change when actions are taken (with probabilities).
- **Rewards**: Immediate feedback from the environment.
  Key property: **Markovian** – the future depends only on the current state and action, not history. Used to formalize reinforcement learning problems.

Bellman Equation
A **fundamental recursive equation** in reinforcement learning that breaks down the value of a state (or state-action pair) into:

- The **immediate reward** received.
- The **discounted value** of future states.
  It expresses that the optimal value now equals the best possible immediate reward plus the optimal value of what follows. This **recursive relationship** enables dynamic programming and many RL algorithms.

Temporal Difference (TD) Learning
A **core RL method** that learns by **bootstrapping** – updating estimates based on other estimates (without waiting for final outcomes).

- **Key idea**: Learn from the difference between predicted and partially observed outcomes.
- **Example**: TD(0) updates the value estimate after each step using the observed reward and the estimated value of the next state.
- **Combines** ideas from Monte Carlo (learning from experience) and dynamic programming (bootstrapping).

Exploration vs. Exploitation
The **fundamental trade-off** in reinforcement learning:

- **Exploitation**: Choose actions known to yield high rewards based on current knowledge.

- **Exploration**: Try new or uncertain actions to discover potentially better outcomes.
  Balancing them is crucial: too much exploitation may miss better strategies; too much exploration may waste time on poor actions.

Deep Q-Learning (DQN)
A **breakthrough RL algorithm** that combines **Q-learning** (a value-based method) with **deep neural networks**.

- **Q-learning**: Learns a function (Q-function) that estimates the expected future reward for each action in each state.
- **Deep Q-Network (DQN)**: Uses a deep neural network to approximate the Q-function for high-dimensional state spaces (like images).
- **Key innovations**: Experience replay (store and sample past transitions) and target networks (stable learning).
  Enables agents to learn from **raw sensory input** (e.g., pixels in games) without hand-engineered features.

2. Describe the difference between Reinforcement Learning and Supervised Learning.